

GRAPH SEARCH

자료구조 스터디 Week14

학생회

 Made with Gamma

INDEX

01

THEORY

02

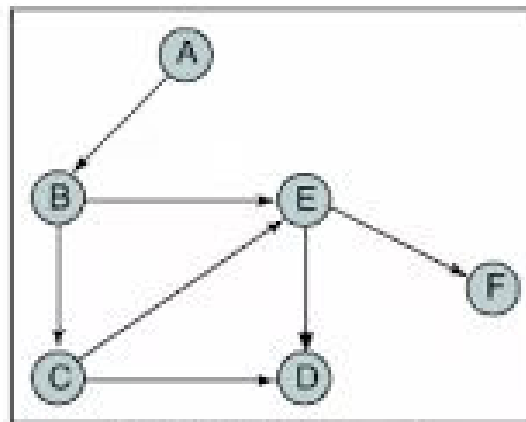
IMPLEMENTATION

03

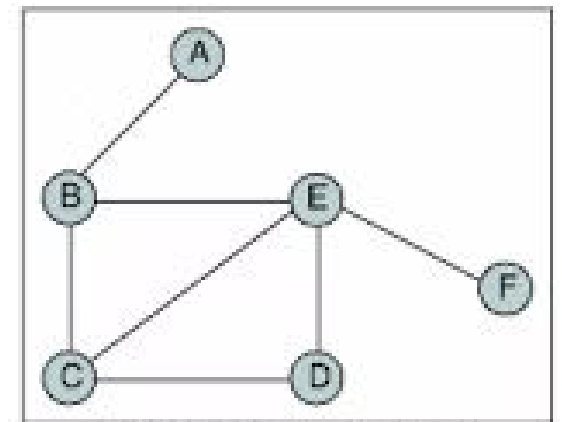
PROBLEM SOLVING

그래프 탐색

그래프 탐색은 하나의 정점으로부터 시작하여 차례대로 모든 정점들을 한 번씩 방문하는 것입니다. DFS(깊이 우선 탐색)와 BFS(너비 우선 탐색)으로 구분할 수 있습니다.



(a) Directed graph



(b) Undirected graph

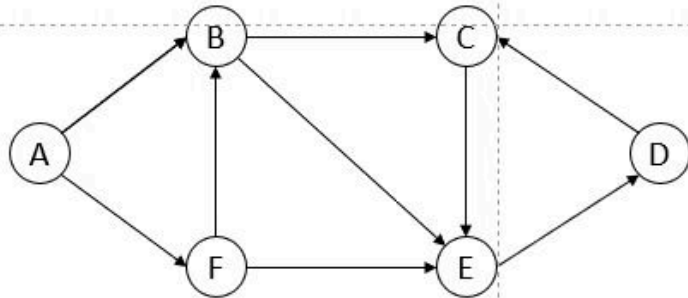
DEPTH FIRST SEARCH

DFS(Depth First Search)

- 현재 정점에서 갈 수 있는 한 끝까지 탐색, 더 이상 방문할 수 있는 정점이 없다면 리턴
- 스택, 재귀를 통해 구현 가능
- 미로 탐색 같은 문제의 해결 기법

DFS(Depth First Search)

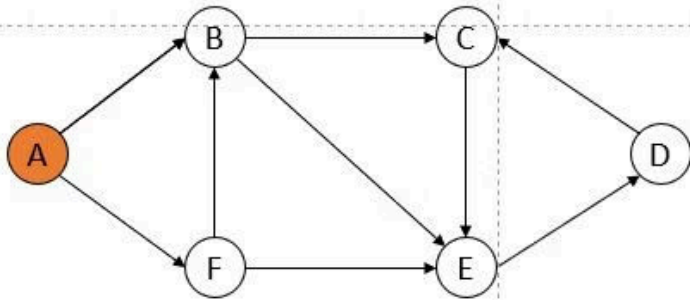
- 방문된 노드
- 방문되지 않은 노드



함수 스택

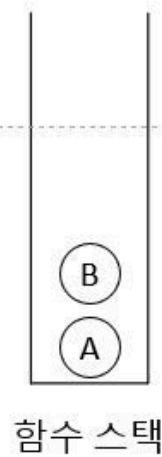
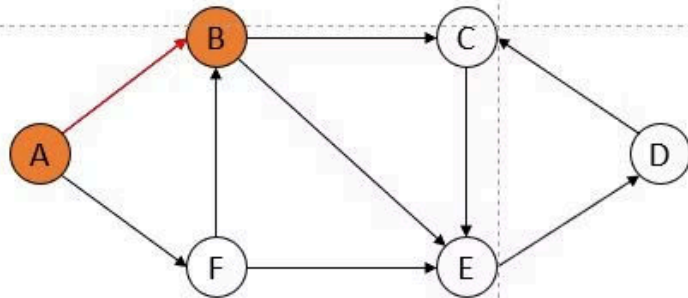
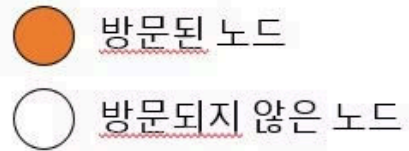
DFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드

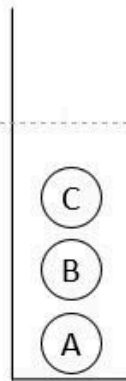
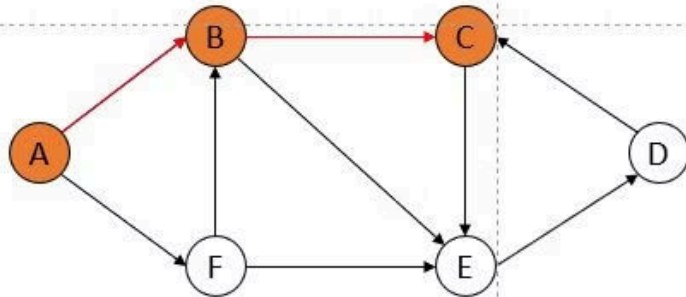
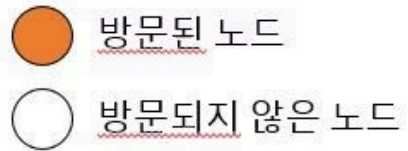


함수 스택

DFS(Depth First Search)

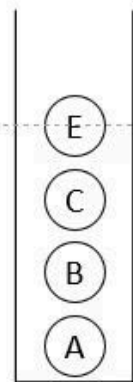
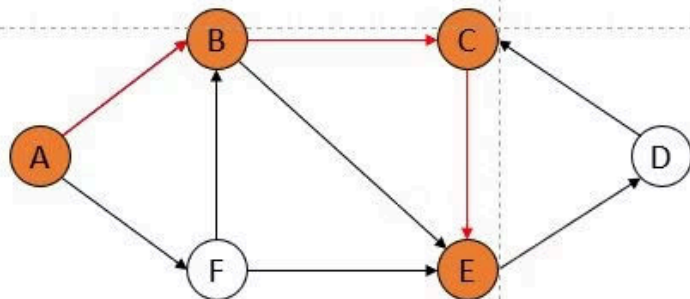
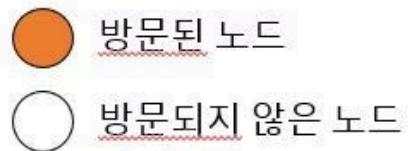


DFS(Depth First Search)



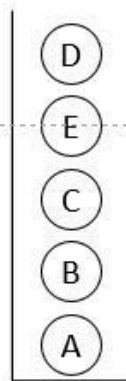
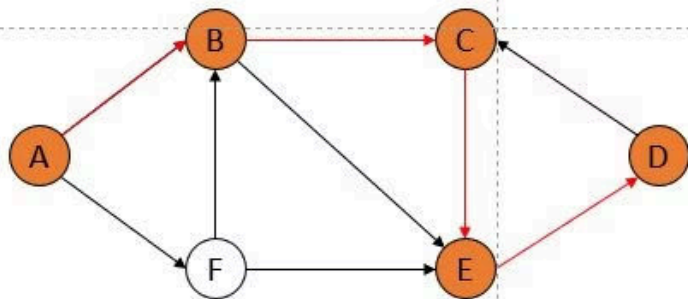
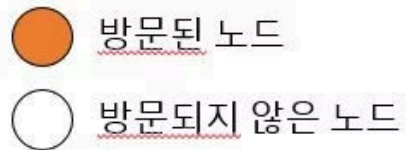
함수 스택

DFS(Depth First Search)



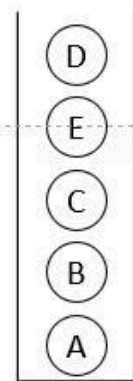
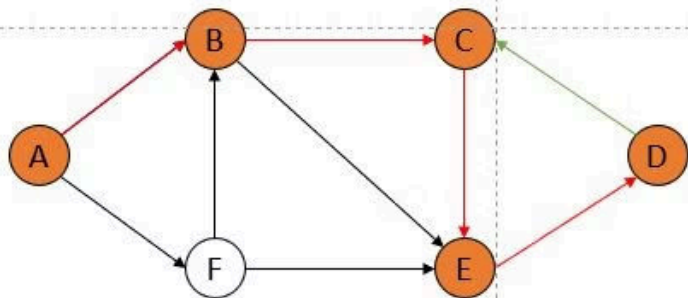
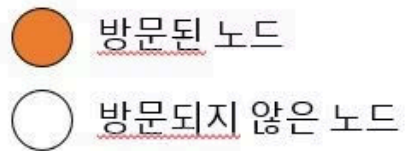
함수 스택

DFS(Depth First Search)



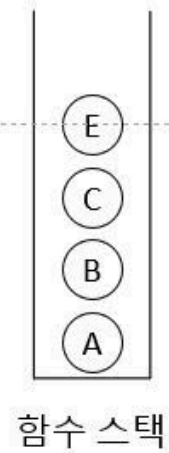
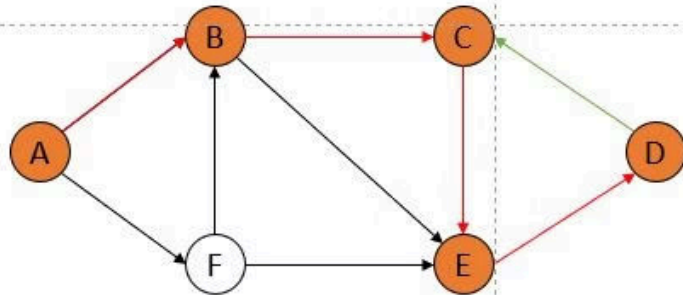
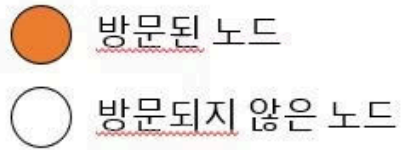
함수 스택

DFS(Depth First Search)

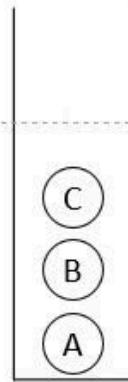
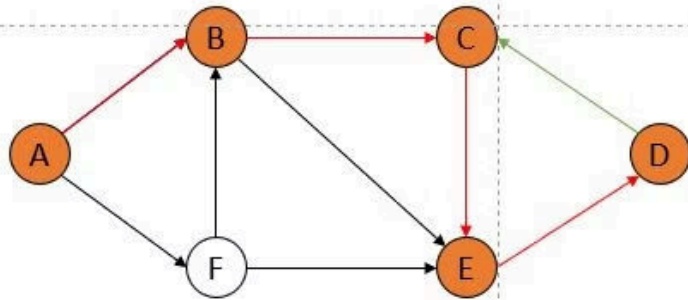
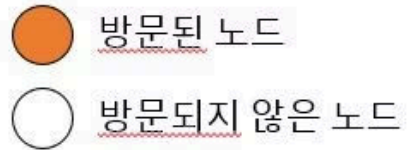


함수 스택

DFS(Depth First Search)



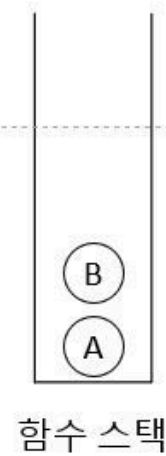
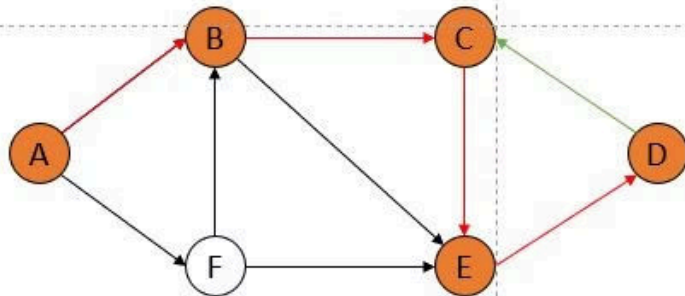
DFS(Depth First Search)



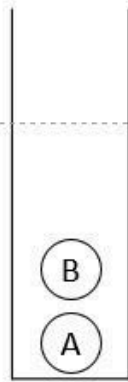
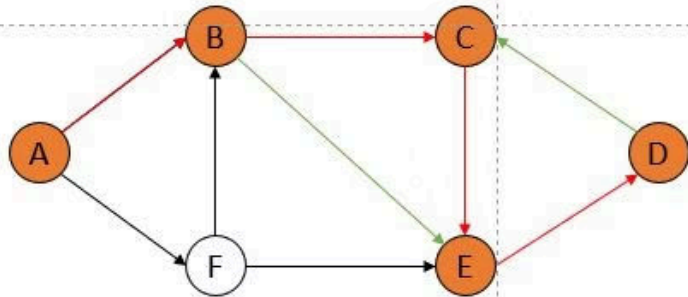
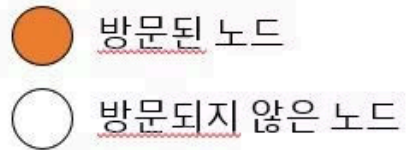
함수 스택

DFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드

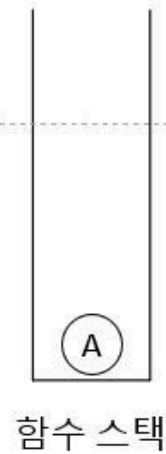
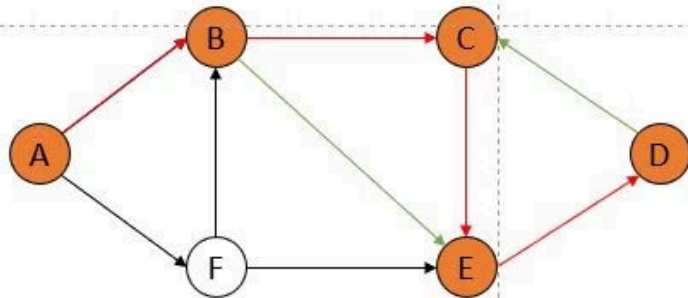
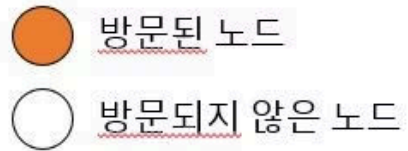


DFS(Depth First Search)



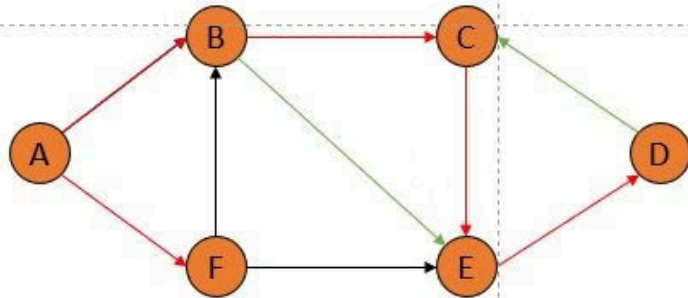
함수 스택

DFS(Depth First Search)



DFS(Depth First Search)

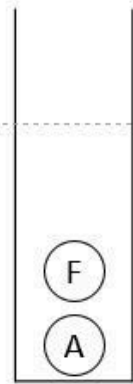
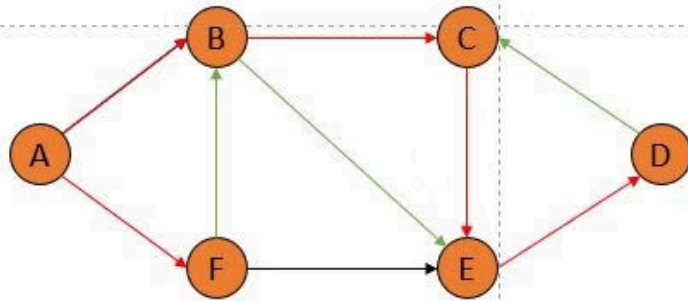
- 방문된 노드
- 방문되지 않은 노드



함수 스택

DFS(Depth First Search)

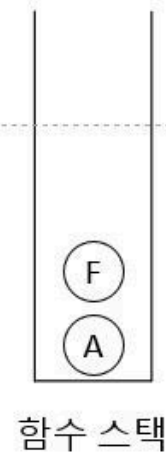
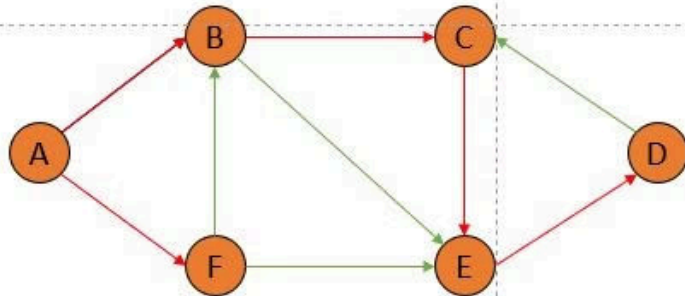
- 방문된 노드
- 방문되지 않은 노드



함수 스택

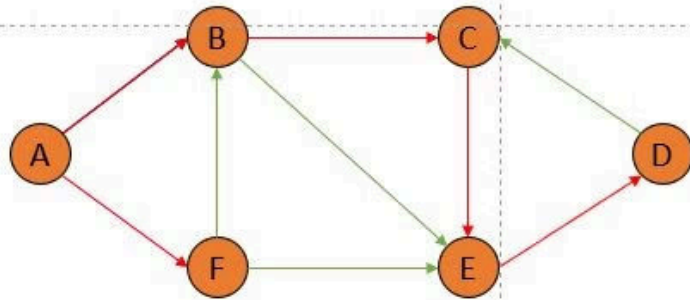
DFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드



DFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드

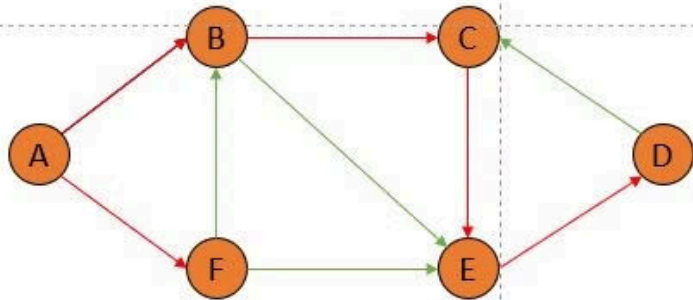


함수 스택

DFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



함수 스택

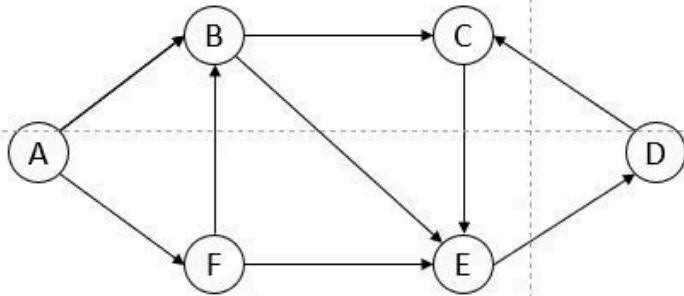
BREADTH FIRST SEARCH

BFS(Depth First Search)

- 현재 정점에서 같은 거리에 있는 정점들 우선 탐색
- 큐를 통해 구현 가능
- 최단 경로 탐색 같은 문제의 해결 기법

BFS(Depth First Search)

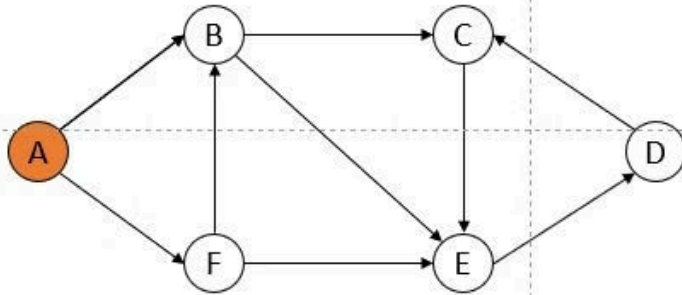
- 방문된 노드
○ 방문되지 않은 노드



큐

BFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드



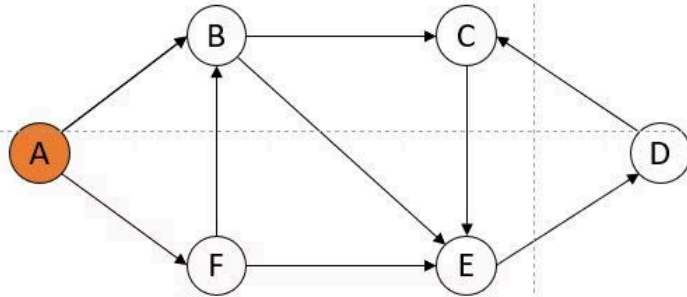
① A

큐

BFS(Depth First Search)

방문된 노드

방문되지 않은 노드

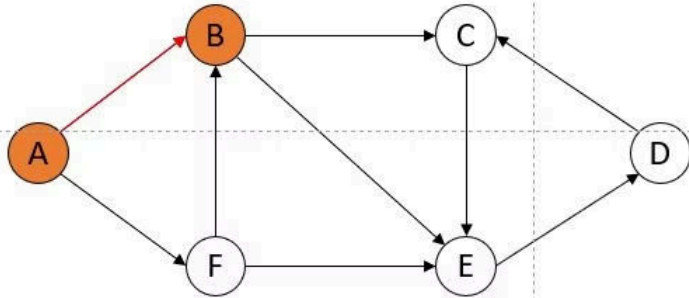


큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



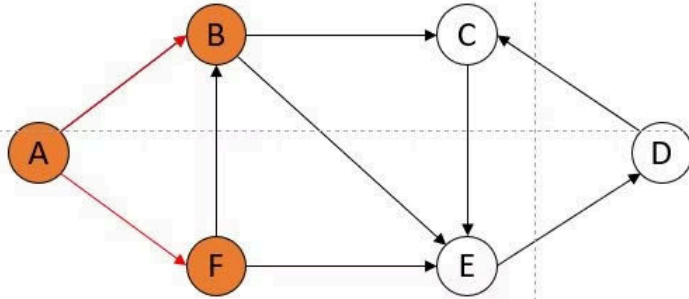
B

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



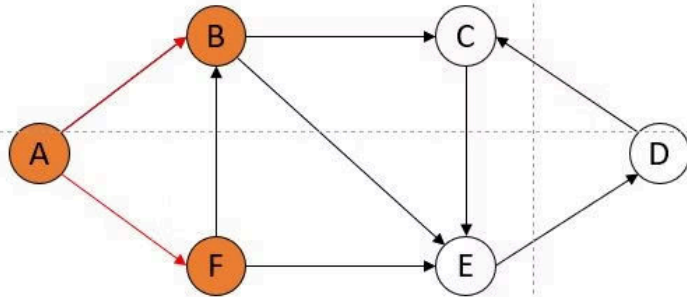
B F

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



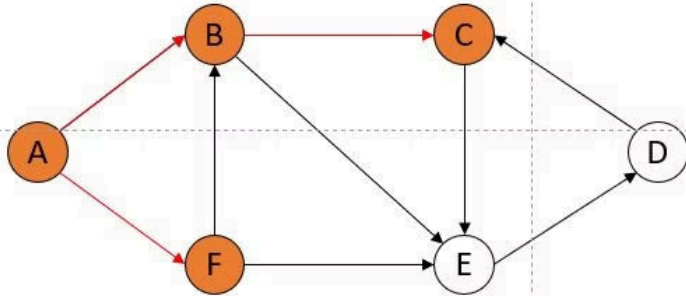
F

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



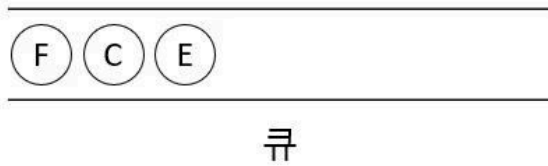
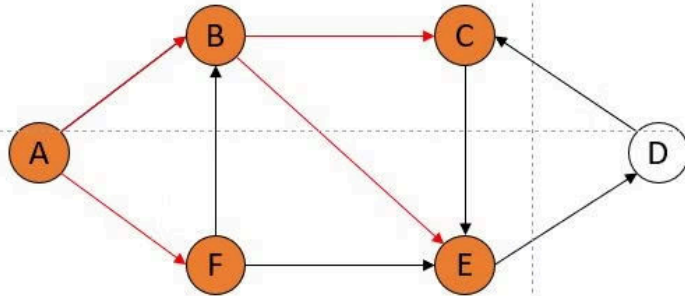
F C

큐

BFS(Depth First Search)

● 방문된 노드

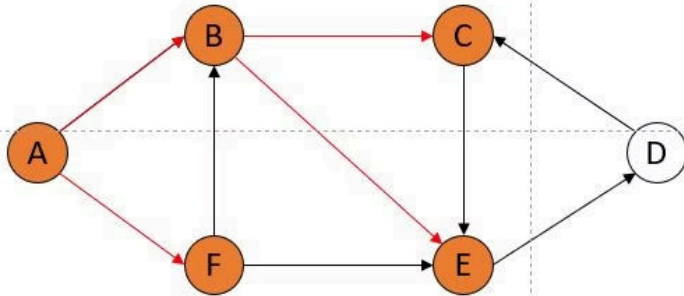
○ 방문되지 않은 노드



BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



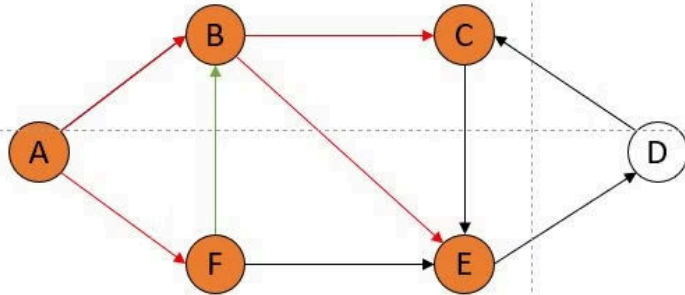
C E

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드

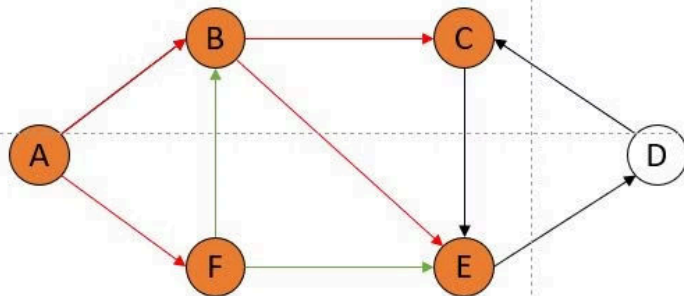


C E

큐

BFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드



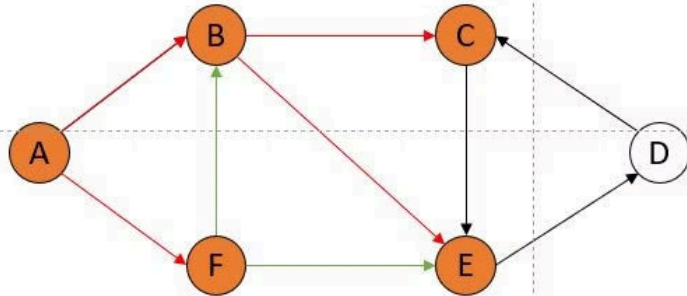
C E

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



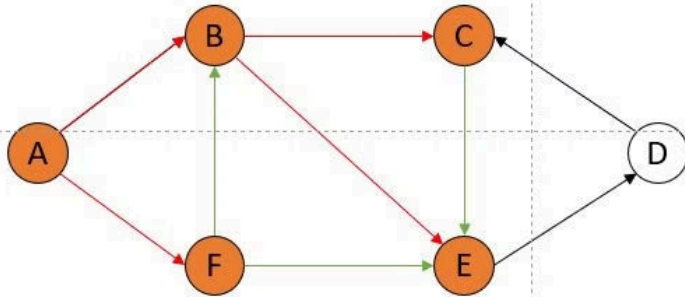
E

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



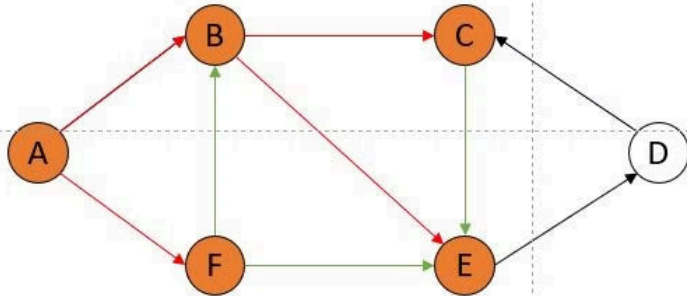
E

큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드

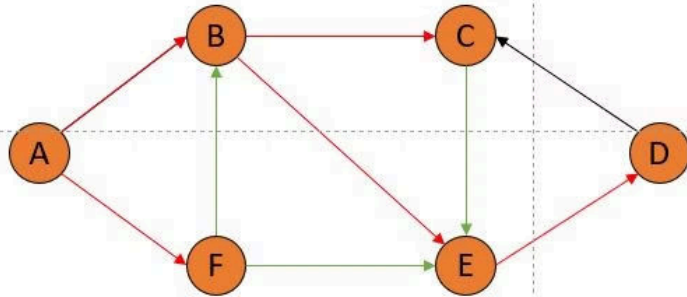


큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드

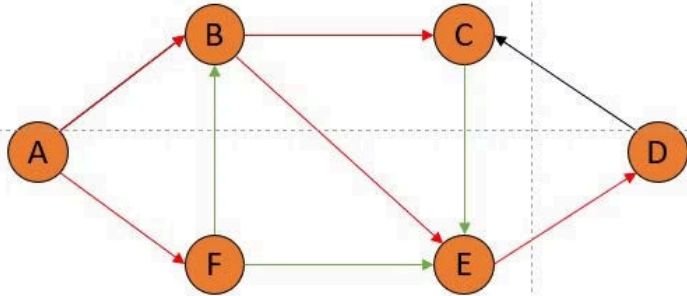


D

큐

BFS(Depth First Search)

- 방문된 노드
- 방문되지 않은 노드

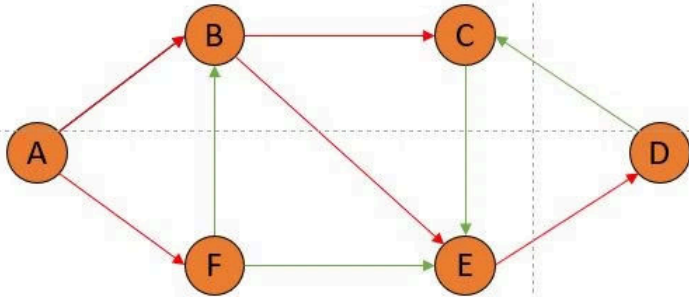


큐

BFS(Depth First Search)

● 방문된 노드

○ 방문되지 않은 노드



큐

IMPLEMENTATION

Tip - 방문할 수 있는 정점 둘 이상인 경우

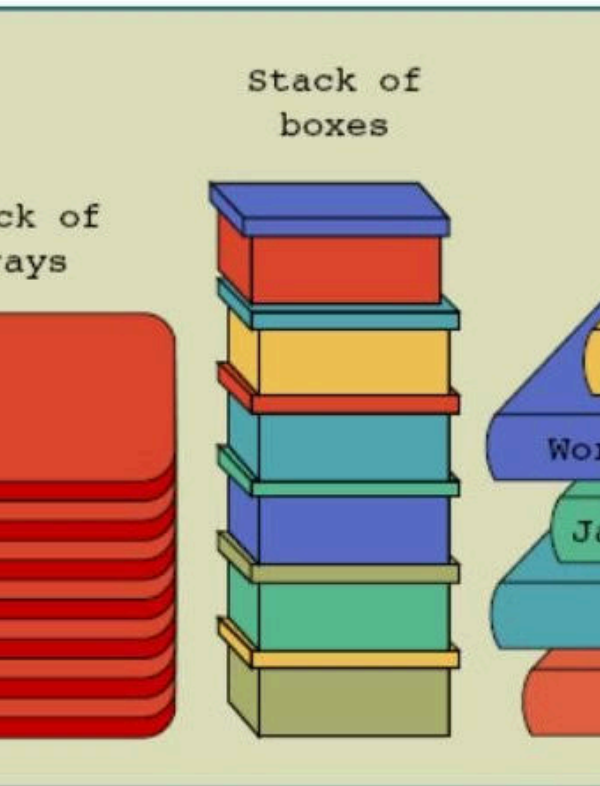
- 번호 작은 정점 우선

인접 행렬 표현법(Adjacency Matrix Representation)으로 그래프 구현

- 해당 정점과 먼저 간선으로 연결된 정점 우선

인접 리스트 표현법(Adjacency List Representation)으로 그래프 구현

Stacks



Stacks of stacks

그래프 탐색의 구현 방법

1

DFS(Depth First Search)

깊이 우선 탐색

2

BFS(Breadth First Search)

너비 우선 탐색

DEPTH FIRST SEARCH

DFS

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  vector<int> graph[1001];
6  bool visited[1001];
7
8  void setup() {
9      for (int i = 0; i < 1001; i++) {
10         visited[i] = false;
11     }
12 }
13
14 void DFS(int a) {
15     int cur = a;
16
17     for (int i = 0; i < graph[cur].size(); i++) {
18         int next = graph[cur][i];
19
20         if (visited[next])
21             continue;
22
23         visited[next] = true;
24         DFS(next);
25     }
26 }
```

BREADTH FIRST SEARCH

BFS

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  vector<int> graph[1001];
7  bool visited[1001];
8  queue<int> q;
9
10 void setup() {
11     for (int i = 0; i < 1001; i++) {
12         visited[i] = false;
13     }
14 }
15
16 void BFS(int a) {
17     while (!q.empty()) {
18         int cur = q.front(); q.pop();
19
20         for (int i = 0; i < graph[cur].size(); i++) {
21             int next = graph[cur][i];
22
23             if (visited[next])
24                 continue;
25
26             visited[next] = true;
27             q.push(next);
28         }
29     }
30 }
```


PROBLEM SOLVING

**THANK'S FOR
WATCHING**